

Reels Links to Content

Master Product, UX, Architecture and Implementation Specification

Project Source of Truth

27 September 2026

Contents

Document Purpose	4
1. Non-Negotiable Product Definition	4
2. Product Principles	4
3. User Personas and Jobs to Be Done	5
3.1 Primary user	5
3.2 Primary jobs	5
4. Scope	5
4.1 MVP scope	5
4.2 Post-MVP scope	6
4.3 Explicitly out of scope for v1	6
5. Critical Terminology	6
6. End-to-End User Experience	6
6.1 Fastest path: share from another app	6
6.2 Manual path	7
6.3 Later retrieval	7
7. Information Architecture	7
7.1 Home	7
7.2 Inbox	7
7.3 Library	8
7.4 Ask	8
7.5 Settings	8
8. UI/UX Wireframes	8
8.1 Share capture sheet	8
8.2 Inbox	9
8.3 Content detail	9
8.4 Library search	10
9. UX State Model	10
10. Technical Architecture	10
10.1 Recommended baseline stack	10
10.2 Logical architecture	11
10.3 Architectural rule: dependency direction	11
11. Platform Adapter Contract	11
12. Platform Integration Policy	12
12.1 Extraction priority	12

13. Data Model	12
13.1 Core tables	12
14. Capture Pipeline	15
14.1 Capture sequence	15
14.2 Duplicate behavior	15
15. Processing Pipeline	15
15.1 Metadata stage	15
15.2 Media acquisition stage	16
15.3 Transcription stage	16
15.4 OCR stage	16
15.5 AI enrichment stage	16
16. Search Architecture	16
16.1 Phase A: metadata search	16
16.2 Phase B: full-text search	16
16.3 Phase C: semantic search	17
17. Ask-My-Library / RAG Design	17
18. AI Prompting and Output Contracts	17
19. Privacy, Security and Rights	18
19.1 Security requirements	18
19.2 Platform and copyright boundaries	18
20. Reliability and Error Handling	18
20.1 Fundamental rule	18
20.2 Error classes	18
21. Observability and Auto-Logging	19
22. API Surface - Initial Draft	19
Authentication	19
Content	19
Collections	19
Search	20
Internal worker endpoints/services	20
23. Detailed Implementation Roadmap	20
Phase 00 - Repository, Standards and Product Baseline	20
Phase 01 - Authentication and Base Application Shell	20
Phase 02 - Capture MVP	21
Phase 03 - Collections, Tags and Library	21
Phase 04 - Processing Job Framework	21
Phase 05 - Adapter Framework + Generic Web Metadata	21
Phase 06 - Official Platform Metadata Integrations	22
Phase 07 - Object Storage and Controlled Media Pipeline	22
Phase 08 - Transcription	22
Phase 09 - OCR / Visual Text Extraction	23
Phase 10 - AI Enrichment	23
Phase 11 - Full-Text Search	23
Phase 12 - Semantic Search	23
Phase 13 - Ask My Library	24
Phase 14 - Knowledge Connections	24
Phase 15 - Export, Portability and Recovery	24
Phase 16 - Performance, Cost and Production Hardening	24
24. Recommended MVP Cut Line	25

25. Testing Strategy	25
25.1 Unit tests	25
25.2 Integration tests	25
25.3 End-to-end tests	25
25.4 Platform contract tests	26
26. Acceptance Criteria Examples	26
Capture story	26
Duplicate story	26
Failure story	26
Ask story	26
27. Data Retention Strategy	26
28. Cost-Control Design	26
29. Accessibility and Usability	27
30. Visual Design Direction	27
31. Versioning and Migration Rules	27
32. AI Coding Agent Guardrails - Mandatory	27
33. Per-Phase Agent Execution Template	28
34. Architecture Decision Records Required	28
35. Suggested Repository Structure	29
36. Example Content Item JSON	29
37. Future Expansion Without Architectural Rewrite	30
38. Risks and Mitigations	30
Risk: platform scraping breaks frequently	30
Risk: product becomes an expensive AI pipeline before user value is proven	30
Risk: media storage cost explodes	30
Risk: AI summaries make unsupported claims	30
Risk: semantic search feels inaccurate	30
Risk: users cannot trust processing state	30
Risk: unofficial extraction violates terms or becomes operationally fragile	31
39. Product Success Metrics	31
40. Launch Checklist	31
41. Immediate Next Build Sequence	31
42. Current External-Integration Verification Notes	32
43. Change Control	32
Version History	32
Appendix A - Definition of Done for Any Phase	32
Appendix B - Agent Handoff Summary	32
Appendix C - One-Sentence Guardrail	33
Appendix D - Verified Official Reference Links (27 September 2026)	33

Document Purpose

This document is the implementation source of truth for **Reels Links to Content**, a personal content-capture and AI knowledge-library system. It is written so that a developer or AI coding agent can continue implementation without inventing requirements, silently changing product behavior, or confusing platform limitations with product requirements.

The system begins with a simple user action: while viewing a reel, short video, social post, or webpage, the user shares or pastes its URL into the application. The system stores the link immediately, identifies the source, obtains whatever metadata/content it is legitimately and technically able to access, processes the content into useful knowledge, organizes it, and makes it searchable later.

The product is **not** primarily a social-media scraper. It is a **personal knowledge capture system with pluggable content extractors**.

1. Non-Negotiable Product Definition

1.1 Core problem

Useful information is increasingly discovered inside Instagram Reels, TikTok videos, YouTube Shorts, Facebook posts/reels, X posts, Reddit threads, articles, and other web content. Native “Save” or bookmark features preserve the item but usually fail at retrieval, cross-platform organization, transcript search, synthesis, personal notes, and long-term knowledge reuse.

The product solves this by turning a shared link into a durable **Content Item** in a personal knowledge library.

1.2 Core promise

The user should be able to save useful content in seconds and later find, understand, group, summarize, compare, and ask questions about it even when the original source is buried among hundreds or thousands of saved items.

1.3 Product statement

A personal AI content library that captures links shared from social media and the web, extracts and understands the available content, automatically organizes it, and makes the resulting knowledge searchable and conversational later.

1.4 What the product is not

The system is not:

- a bulk social-media downloader;
- a piracy tool;
- a replacement social network;
- a guaranteed full-content extractor for every platform;
- dependent on a single unofficial scraper;
- an autonomous reposting tool;
- a system that claims extraction succeeded when only metadata was available.

The application must distinguish clearly between **link saved**, **metadata extracted**, **media accessible**, **transcription complete**, **AI analysis complete**, and **failed/partial processing**.

2. Product Principles

1. **Capture first, process second.** Saving the URL must be fast and reliable even when processing is delayed or unavailable.
2. **Never lose a link because extraction failed.** The original URL remains the canonical fallback.
3. **Platform adapters are replaceable.** Instagram, TikTok, YouTube, Facebook, X, Reddit, and generic-web integrations must not contaminate core domain logic.
4. **Progressive enrichment.** A Content Item can become more useful over time as metadata, transcript, OCR, summary, tags, embeddings, and notes are added.

5. **Provenance matters.** The user must be able to see where extracted text and AI-generated information came from.
6. **No fabricated extraction.** If a transcript is unavailable, the UI and database must say so.
7. **User control over retention.** Media retention, transcripts, AI outputs, notes, and deletion must be explicit.
8. **Search is a first-class product, not an afterthought.** Data models must support exact, filtered, full-text, and semantic search.
9. **Mobile capture must be frictionless.** On Android, the application should appear as a share target for text/URLs.
10. **AI is an enrichment layer, not the system of record.** Canonical URLs, metadata, user notes, and verified extracted text remain separate from generated summaries.

3. User Personas and Jobs to Be Done

3.1 Primary user

A knowledge-heavy user who frequently discovers useful short-form content but cannot reliably retrieve it later.

Common domains may include technology, programming, business, research, engineering, learning, health, productivity, professional work, and personal interests.

3.2 Primary jobs

The user wants to:

- save a reel/post/article without interrupting what they are doing;
- remember why it was saved;
- find it later using ordinary language;
- search inside spoken content, captions, and notes;
- group related items into projects or collections;
- ask questions across multiple saved items;
- identify repeated advice or conflicting claims;
- turn saved content into actionable knowledge rather than an archive of forgotten links.

4. Scope

4.1 MVP scope

The MVP must support:

- account/authentication;
- Android share-to-app URL capture;
- manual URL paste in mobile/web UI;
- URL normalization and duplicate detection;
- supported-platform detection;
- immediate creation of a Content Item;
- processing queue and visible processing state;
- basic metadata retrieval where available;
- title, source, creator, thumbnail where available;
- user notes and “why I saved this”;
- collections;
- favorites;
- tags;
- basic search and filtering;
- retryable extractor architecture;
- audit/error logs at backend level;
- deletion and retention controls.

4.2 Post-MVP scope

Later phases may add:

- audio/video acquisition where allowed;
- speech-to-text transcription;
- frame OCR;
- richer AI summaries and key-point extraction;
- entity extraction;
- embeddings and semantic search;
- cross-item chat/RAG;
- knowledge connections and clusters;
- browser extension;
- iOS share extension;
- email/Telegram/WhatsApp capture channels where technically appropriate;
- export/import;
- collaborative/shared collections;
- advanced analytics.

4.3 Explicitly out of scope for v1

Do not build these unless a future specification explicitly promotes them:

- automated reposting;
- social engagement automation;
- follower growth tools;
- bypassing platform access controls;
- downloading DRM/restricted/private content without authorization;
- public social profiles inside this product;
- creator analytics suite;
- full multi-user team collaboration;
- recommendation feed designed for endless consumption.

5. Critical Terminology

Content Item - the core saved object representing one URL/source item.

Capture - the act of creating a Content Item from a shared/pasted URL.

Extractor - a platform-specific component that attempts to retrieve permitted metadata or content.

Enrichment - any additional processing after capture: transcript, OCR, summary, tags, embeddings, etc.

Artifact - a stored output of processing, such as transcript text, OCR text, summary JSON, or thumbnail.

Collection - user-managed grouping of Content Items.

Inbox - newly captured items not yet intentionally organized, including items still processing.

Processing Job - a durable backend job representing one stage of extraction/enrichment.

Provenance - information describing which source or process produced a field/output.

Canonical URL - normalized stable representation of a source URL used for deduplication and lookup.

6. End-to-End User Experience

6.1 Fastest path: share from another app

1. User watches a reel/post/video.
2. User taps **Share**.
3. User selects **Reels Links to Content**.
4. A lightweight capture screen appears.
5. URL is parsed immediately.
6. User can tap **Save** immediately, optionally adding:

- reason for saving;
 - collection;
 - note;
 - favorite flag.
7. UI confirms **Saved**.
 8. Backend processing continues independently of the capture UI.
 9. The item appears in Inbox with a processing state.

The capture UX must not require the user to wait for transcription or AI processing.

6.2 Manual path

Home > Add > Paste URL > preview if available > optional note/collection > Save.

6.3 Later retrieval

User can locate content by:

- exact words;
- creator/platform;
- date saved;
- collection;
- tags;
- favorite;
- processing status;
- semantic query;
- question across library.

7. Information Architecture

Primary navigation for the mobile application:

1. **Home**
2. **Inbox**
3. **Library**
4. **Ask**
5. **Profile/Settings**

A floating or prominent **Add** action is available from Home, Inbox, and Library.

7.1 Home

Purpose: orientation and fast continuation.

Contents:

- search bar;
- Add/Save link action;
- recent saves;
- items needing attention;
- recent collections;
- optional insights such as “12 items saved this week”;
- no infinite entertainment feed.

7.2 Inbox

Purpose: capture triage.

Segments/filters:

- Processing;
- Ready;
- Needs attention;
- Unsorted;

- Failed.

Each item card should show:

- thumbnail or platform icon;
- title/caption snippet;
- platform;
- creator if known;
- saved time;
- processing status;
- collection if assigned;
- favorite indicator;
- retry action only when relevant.

7.3 Library

Purpose: durable knowledge browsing.

Views:

- All items;
- Collections;
- Tags;
- Favorites;
- Platforms;
- Recently saved;
- Recently viewed.

7.4 Ask

Purpose: retrieval-assisted conversation over saved content.

The Ask screen is disabled or clearly labeled “coming later” until semantic indexing/RAG is actually implemented. Do not ship a fake chat screen that answers without grounding in the user’s library.

7.5 Settings

At minimum:

- account;
- storage/retention;
- AI processing preferences;
- download/media retention preference;
- privacy;
- export;
- delete account/data;
- connected platform authorizations;
- diagnostics/log submission toggle if implemented;
- app version and backend version.

8. UI/UX Wireframes

These are structural wireframes, not final visual designs.

8.1 Share capture sheet

```
+-----+
| Save to Library |
+-----+
| [thumbnail] Instagram Reel |
| @creator          |
| Short caption/title... |
|                   |
```

Why are you saving this?
[Idea] [Learn] [Project]
[Research] [Other]
Note (optional)
[_____]
Collection
[Inbox v]
[Cancel] [Save]

- Rules:
- Save remains enabled if the URL is valid even if preview retrieval fails.
 - Preview loading must never block capture.
 - “Why saved” presets are optional metadata, not required fields.

8.2 Inbox

Inbox Search
[All] [Processing] [Failed]
[img] Borehole sizing tips
YouTube Short
Processing transcript 62%
[img] AI SaaS pricing lesson
TikTok - @creator
Ready
[ico] Private/Unavailable item
Instagram
Link saved - extraction
unavailable

8.3 Content detail

< Back ...
[thumbnail / embed if permitted]
Title / caption
Creator - Platform - Saved date
[Open Original] [Favorite]
Summary
...
Key points
- ...
- ...
Transcript
[Search within transcript]
...
My note
...

```
|-----|
| Tags / Collection |
+-----+
```

AI sections must carry a subtle label such as “AI-generated”. Extracted/transcribed source text should be visually distinct from generated interpretation.

8.4 Library search

```
+-----+
| Search your library |
| [storage advice for SautiScript] |
+-----+
| Filters: Platform Date Tag |
|           Collection Type |
+-----+
| 18 results |
|
| [item] Object storage vs DB |
|           matched transcript |
|
| [item] Async media processing |
|           matched summary + note |
+-----+
```

9. UX State Model

Every Content Item must have an explicit state. Avoid a single overloaded “status” field if stage-specific state is needed.

Recommended high-level states:

- captured
- metadata_pending
- metadata_ready
- content_pending
- content_partial
- content_ready
- enrichment_pending
- ready
- failed_retryable
- failed_terminal
- source_unavailable
- deleted

The UI may collapse these to friendlier labels, but backend states must remain unambiguous.

10. Technical Architecture

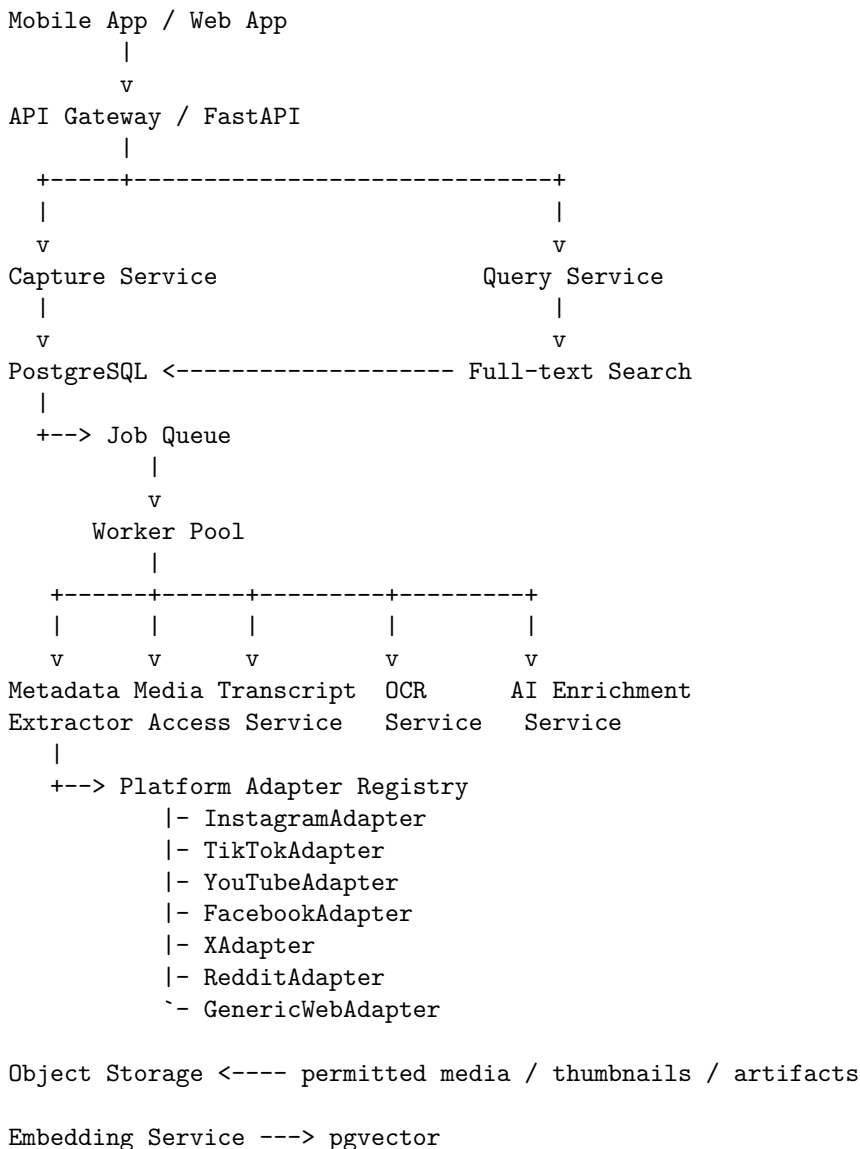
10.1 Recommended baseline stack

This is a pragmatic baseline, not an eternal requirement:

- **Mobile:** Flutter
- **Web/Admin:** React/Next.js or equivalent
- **Backend API:** Python FastAPI
- **Database:** PostgreSQL
- **Managed backend option:** Supabase for PostgreSQL, Auth, Storage, and pgvector
- **Queue:** Redis + worker framework, or durable database-backed queue initially
- **Object storage:** Supabase Storage or S3-compatible service
- **AI/transcription providers:** provider abstraction; never hard-code one vendor into domain logic
- **Embeddings:** provider abstraction + pgvector

- **Observability:** structured logs, job events, request IDs, error aggregation

10.2 Logical architecture



10.3 Architectural rule: dependency direction

Core domain logic must depend on interfaces, not platform SDKs.

Bad:

```
ContentService -> TikTok SDK directly
```

Good:

```
ContentService -> ExtractorInterface -> TikTokAdapter
```

11. Platform Adapter Contract

Each adapter should expose a normalized interface similar to:

```

can_handle(url) -> bool
normalize_url(url) -> canonical_url
extract_metadata(url, auth_context?) -> MetadataResult
get_embed(url) -> EmbedResult | NotSupported
  
```

```
acquire_media(url, auth_context?) -> MediaResult | NotPermitted | NotSupported
health_check() -> AdapterHealth
```

`MetadataResult` should contain explicit provenance and missing-field indicators.

Example normalized metadata fields:

- platform;
- platform_item_id;
- canonical_url;
- creator_name;
- creator_handle;
- title;
- description/caption;
- thumbnail_url;
- published_at;
- duration_seconds;
- media_type;
- embed_html/embed_url if permitted;
- extraction_method;
- extracted_at;
- raw_provider_payload reference (optional, access-controlled).

Adapters must **not** invent fields. Null is valid.

12. Platform Integration Policy

Platform capabilities change. Before implementing or modifying any adapter, the developer/AI agent must verify current official documentation and terms.

Current implementation anchors as of 27 September 2026:

- Android officially supports receiving shared text/URLs using share intents and intent filters. The receiving application should register appropriate `ACTION_SEND` handling rather than depend on clipboard hacks.
- TikTok provides official content-display capabilities including oEmbed and a Display API with user authorization/scopes for relevant user/video data.
- YouTube provides an official Data API for video/channel metadata, with API-key/OAuth requirements depending on operation.

These statements are architectural anchors only. Exact endpoints, quotas, permissions, review requirements, allowed retention, and terms must be re-checked at implementation time.

12.1 Extraction priority

For each platform, use this preference order:

1. official API explicitly suitable for the use case;
2. official oEmbed/embed metadata;
3. public page metadata permitted by terms and robots/policies;
4. user-authorized access;
5. carefully reviewed browser extraction only if lawful, permitted, maintainable, and necessary;
6. link-only fallback.

A failed extractor never means a failed capture.

13. Data Model

13.1 Core tables

users

- id
- email / auth identity

- created_at
- updated_at
- settings_json

content_items

- id UUID
- user_id
- original_url
- canonical_url
- platform
- platform_item_id nullable
- content_type
- title nullable
- caption nullable
- creator_name nullable
- creator_handle nullable
- published_at nullable
- thumbnail_url nullable
- duration_seconds nullable
- status
- source_availability_status
- saved_reason nullable
- user_note nullable
- is_favorite boolean
- primary_collection_id nullable
- captured_at
- last_processed_at nullable
- created_at
- updated_at
- deleted_at nullable

Unique/deduplication strategy should normally be user-scoped canonical URL or (user_id, platform, platform_item_id) when reliable.

collections

- id
- user_id
- name
- description nullable
- icon/color metadata optional
- created_at
- updated_at

content_collections

Use if multi-collection membership is desired. If v1 allows only one collection per item, this table can be postponed, but the API should not make a future migration impossible.

tags

- id
- user_id or global namespace decision
- name
- normalized_name

content_tags

- content_item_id
- tag_id
- origin (user, ai, import)

- confidence nullable

artifacts

Stores references to processing outputs.

- id
- content_item_id
- artifact_type (`transcript`, `ocr`, `summary`, `key_points`, `thumbnail`, etc.)
- storage_location or text/json value according to type
- provider
- model nullable
- version
- provenance_json
- created_at

processing_jobs

- id
- content_item_id
- job_type
- state
- priority
- attempt_count
- max_attempts
- progress_percent nullable
- stage_message nullable
- lease/heartbeat fields if workers claim jobs
- scheduled_at
- started_at nullable
- completed_at nullable
- failed_at nullable
- error_code nullable
- sanitized_error_message nullable
- created_at

processing_events

Append-only operational timeline.

- id
- job_id
- content_item_id
- event_type
- stage
- status
- message sanitized
- metadata_json sanitized
- created_at

embeddings

Can be a separate table or associated with chunk records.

- id
- content_item_id
- artifact_id nullable
- chunk_index
- chunk_text/hash
- embedding vector
- embedding_model
- created_at

search_chunks

Recommended once semantic search exists.

- id
- content_item_id
- source_type (caption, transcript, ocr, note, summary)
- source_artifact_id nullable
- chunk_index
- text
- token_count
- tsvector/full-text field
- created_at

14. Capture Pipeline

14.1 Capture sequence

Receive shared text

- > extract URL(s)
- > validate URL syntax
- > identify adapter
- > normalize canonical URL
- > check duplicate
- > create or return existing Content Item
- > persist user's optional note/reason/collection
- > enqueue metadata extraction
- > return success immediately

14.2 Duplicate behavior

If the same user saves the same item twice:

- do not silently create unnecessary duplicate records;
- show “Already saved”;
- allow updating note, collection, favorite, or saved-reason;
- optionally record a **saved_again_at** event for future analytics.

Different users may save the same public URL independently.

15. Processing Pipeline

Recommended stages:

CAPTURED

- > NORMALIZED
- > METADATA_EXTRACTED
- > CONTENT_ACCESS_CHECK
- > MEDIA_ACQUIRED (optional)
- > TRANSCRIBED (optional)
- > OCR_COMPLETE (optional)
- > ENRICHED
- > INDEXED
- > READY

Each stage must be idempotent where practical. Re-running a job should not duplicate artifacts or corrupt state.

15.1 Metadata stage

Produces canonical metadata using platform adapter.

15.2 Media acquisition stage

Only run where permitted and necessary. Store media in object storage, never as large blobs inside PostgreSQL.

Retention strategy should support:

- no media retention;
- temporary media retention until transcription/OCR completes;
- user-selected long-term retention when allowed.

15.3 Transcription stage

Input: accessible audio/video.

Output:

- transcript text;
- language;
- optional segments/timestamps if genuinely returned;
- model/provider metadata;
- confidence metadata where available.

Never fabricate timestamps.

15.4 OCR stage

Use selectively, not on every video frame. Candidate approaches:

- periodic key frames;
- scene-change frames;
- text-dense frame detection;
- creator slides/screenshots.

Store OCR output separately from transcript.

15.5 AI enrichment stage

Possible outputs:

- concise summary;
- detailed summary;
- key points;
- topics;
- suggested tags;
- people/organizations/products/tools mentioned;
- action items;
- claims/questions worth verifying;
- suggested collection.

Generated outputs must identify themselves as generated and must not overwrite source text.

16. Search Architecture

Implement search progressively.

16.1 Phase A: metadata search

Search title, caption, creator, note, tags, collection.

16.2 Phase B: full-text search

Index transcript, OCR, captions, notes, summaries.

16.3 Phase C: semantic search

Chunk source material and create embeddings.

Search ranking should combine:

- vector similarity;
- keyword/full-text score;
- user note match boost;
- title match boost;
- recency only as a modest factor;
- collection/filter constraints.

The user should be able to see **why** an item matched: e.g. “matched transcript” or “matched your note”.

17. Ask-My-Library / RAG Design

The chat feature must be grounded in saved content.

Query flow:

User question

- > normalize query
- > apply requested filters
- > hybrid retrieval
- > rank chunks
- > build evidence set
- > LLM answer
- > return citations to Content Items/chunks

Rules:

- answer from retrieved library evidence when the mode is “Ask my library”;
- clearly separate library evidence from optional external-web research;
- every substantive library-derived statement should link back to the saved item(s);
- if evidence is insufficient, say so;
- do not make an answer look grounded merely because a related item exists.

18. AI Prompting and Output Contracts

AI calls must use machine-readable schemas where possible.

Example enrichment response:

```
{
  "summary_short": "...",
  "summary_detailed": "...",
  "key_points": [...],
  "topics": [...],
  "suggested_tags": [...],
  "entities": [
    {"name": "...", "type": "tool", "evidence": "..."}
  ],
  "action_items": [...],
  "uncertainties": [...]}
}
```

The application validates schemas before persistence. Invalid AI output is rejected/retried; it is not patched with invented defaults.

19. Privacy, Security and Rights

19.1 Security requirements

- Row-level access control: users can access only their own private library unless explicit sharing is later implemented.
- Encrypt data in transit.
- Protect service keys server-side; never ship privileged keys inside the mobile app.
- Use signed/temporary URLs for private object storage.
- Sanitize logs to avoid leaking tokens, cookies, private URLs, or full media payloads.
- Rate-limit capture and processing endpoints.
- Validate URLs to reduce SSRF risk.
- Worker media-fetch systems must block private network ranges and unsafe redirects.
- Validate upload/content types.
- Maintain audit events for destructive operations.

19.2 Platform and copyright boundaries

The system must not assume that public visibility equals permission to permanently download/store/reuse media. Each adapter must document:

- official API basis;
- authorization requirement;
- metadata permitted;
- media access status;
- retention constraints;
- attribution/embed requirement;
- known limitations.

When rights are unclear, default to storing the link and user-generated metadata rather than unauthorized media copies.

20. Reliability and Error Handling

20.1 Fundamental rule

Capture success is independent from enrichment success.

Example: Instagram extraction breaks tomorrow. The user can still save the URL and note; the item is marked partial, and processing can be retried after adapter repair.

20.2 Error classes

Use normalized codes such as:

- INVALID_URL
- UNSUPPORTED_PLATFORM
- SOURCE_NOT_FOUND
- SOURCE_PRIVATE
- SOURCE_REMOVED
- AUTH_REQUIRED
- RATE_LIMITED
- PLATFORM_API_ERROR
- EXTRACTION_FAILED
- MEDIA_NOT_PERMITTED
- MEDIA_FETCH_FAILED
- TRANSCRIPTION_FAILED
- OCR_FAILED
- AI_ENRICHMENT_FAILED
- INDEXING_FAILED
- STORAGE_FAILED

User messages should be clear and non-technical; backend logs retain structured diagnostic details.

21. Observability and Auto-Logging

Every request/job should carry a correlation/request ID.

Log fields:

- timestamp;
- request_id;
- user_id hashed/internal ID;
- content_item_id;
- job_id;
- platform;
- adapter;
- stage;
- status;
- duration_ms;
- retry_count;
- normalized error code;
- sanitized provider response code;
- app/backend version.

Dashboard metrics later:

- capture success rate;
- extraction success by platform;
- average processing duration;
- job queue age;
- failure rate by stage;
- provider cost per processed minute/item;
- storage consumption;
- search latency;
- AI token usage.

22. API Surface - Initial Draft

Authentication

Handled by chosen auth provider.

Content

```
POST    /api/content/capture
GET     /api/content
GET     /api/content/{id}
PATCH  /api/content/{id}
DELETE  /api/content/{id}
POST    /api/content/{id}/retry
POST    /api/content/{id}/favorite
```

Collections

```
GET     /api/collections
POST    /api/collections
GET     /api/collections/{id}
PATCH  /api/collections/{id}
DELETE  /api/collections/{id}
POST    /api/collections/{id}/items
DELETE  /api/collections/{id}/items/{content_id}
```

Search

```
GET /api/search?q=...
POST /api/search/semantic
POST /api/ask
```

Do not expose `/ask` until evidence-grounded retrieval exists.

Internal worker endpoints/services

Prefer queue-driven internal services over publicly exposed worker actions.

23. Detailed Implementation Roadmap

Phase 00 - Repository, Standards and Product Baseline

Objective

Create a stable implementation environment before product coding.

Deliverables

- monorepo or clearly separated repositories;
- `/docs` containing this specification;
- environment template `.env.example` without secrets;
- coding conventions;
- migration strategy;
- API error schema;
- structured logging format;
- CI for lint/test/build;
- versioning strategy;
- architecture decision records (`/docs/adr`).

Exit criteria

- clean local setup from README;
- test DB can migrate from zero;
- CI passes;
- no secrets committed;
- this master spec is committed and referenced by agent instructions.

Phase 01 - Authentication and Base Application Shell

Build

- Supabase/Auth or selected auth integration;
- sign-up/login/logout/session refresh;
- mobile navigation shell;
- Home, Inbox, Library, Settings empty states;
- backend user identity enforcement.

Tests

- unauthenticated endpoint rejection;
- one user cannot access another user's records;
- session expiry/refresh handling.

Exit criteria

A real user can log in and navigate the shell; security boundaries are proven by tests.

Phase 02 - Capture MVP

Build

- Android `ACTION_SEND` text/URL receiver;
- incoming shared-text parser;
- manual paste flow;
- URL validation;
- platform detection;
- URL normalization;
- `content_items` schema;
- immediate persistence;
- duplicate detection;
- optional note/reason/favorite/collection-at-capture;
- Inbox list.

Important rule

No platform extraction is required for Phase 02 success.

Exit criteria

From Instagram/TikTok/YouTube/browser, a shared URL can become a durable Content Item within seconds even when the backend cannot fetch the source.

Phase 03 - Collections, Tags and Library

Build

- collection CRUD;
- assign/unassign items;
- manual tags;
- favorites;
- Library views/filters;
- content detail screen;
- edit note/title override without overwriting extracted title.

Exit criteria

The product is already useful as a cross-platform smart bookmark manager before AI.

Phase 04 - Processing Job Framework

Build

- `processing_jobs`;
- worker claim/lease mechanism;
- retry/backoff;
- idempotency keys;
- cancellation where appropriate;
- stale-job recovery;
- `processing_events`;
- Inbox live/poll status updates;
- sanitized error display.

Exit criteria

A dummy test job can fail, retry, recover from worker interruption, and complete without duplication.

Phase 05 - Adapter Framework + Generic Web Metadata

Build

- adapter registry;

- adapter interface;
- GenericWebAdapter;
- OpenGraph/meta title/description/thumbnail extraction where permitted;
- platform-specific URL canonicalizers;
- adapter health checks;
- fixtures/tests.

Exit criteria

Adding a new platform adapter does not require modifying core content-domain logic.

Phase 06 - Official Platform Metadata Integrations

Implement one adapter at a time, beginning with the most reliable/valuable source.

Suggested order:

1. YouTube;
2. TikTok;
3. generic web;
4. Instagram/Meta based on verified current capabilities;
5. Facebook;
6. X;
7. Reddit.

For each adapter create an `ADAPTER_<PLATFORM>.md` containing:

- official documentation links;
- date verified;
- authentication/scopes;
- exact fields used;
- quotas/rate limits;
- storage/retention rules;
- test fixture URLs;
- known failure modes;
- fallback behavior.

Exit criteria

Metadata extraction is deterministic, observable, tested, and does not jeopardize capture.

Phase 07 - Object Storage and Controlled Media Pipeline

Build

- private object-storage bucket;
- media object naming strategy;
- signed URL access;
- retention lifecycle;
- temporary working files;
- per-item media-access policy;
- cleanup worker.

Exit criteria

Permitted media can be processed without database blobs or orphaned storage objects.

Phase 08 - Transcription

Build

- transcription provider interface;
- audio extraction where necessary;
- file-duration/size checks;

- provider job invocation;
- real transcript storage;
- optional true timestamps only when returned;
- transcript search-on-detail;
- failure/retry logic;
- cost metrics.

Exit criteria

Supported media produces traceable transcript artifacts without fake timestamps or silent truncation.

Phase 09 - OCR / Visual Text Extraction

Build

- frame sampling policy;
- OCR provider interface;
- deduplication of repeated on-screen text;
- artifact provenance;
- optional timestamp/frame references.

Exit criteria

OCR improves text-heavy videos without exploding compute/storage cost.

Phase 10 - AI Enrichment

Build

- provider abstraction;
- strict JSON schema;
- prompt versioning;
- summaries;
- key points;
- tag suggestions;
- entities;
- suggested collections;
- AI-generated labeling in UI;
- reprocess with new prompt version.

Exit criteria

Generated information is separated from source artifacts and can be reproduced/versioned.

Phase 11 - Full-Text Search

Build

- search_chunks;
- PostgreSQL full-text indexes;
- unified result ranking;
- filters;
- match-source display.

Exit criteria

User can retrieve an item by a phrase spoken in a processed video.

Phase 12 - Semantic Search

Build

- chunking policy;

- embedding provider abstraction;
- pgvector;
- embedding version/model tracking;
- hybrid search;
- re-index jobs.

Exit criteria

Conceptual searches retrieve relevant items even when exact wording differs.

Phase 13 - Ask My Library

Build

- retrieval pipeline;
- evidence window;
- grounded answer generation;
- citations to Content Items/chunks;
- filters by collection/date/platform;
- “insufficient evidence” behavior;
- conversation history policy.

Exit criteria

Every answer can be traced to the user’s stored evidence; hallucinated citations are rejected.

Phase 14 - Knowledge Connections

Build

- related-item recommendations;
- topic clusters;
- recurring themes;
- contradictions/alternative claims presentation;
- “you saved X items about...” insights;
- collection suggestions.

Exit criteria

Insights are evidence-backed and link to the items used.

Phase 15 - Export, Portability and Recovery

Build

- JSON/CSV export;
- Markdown export;
- collection export;
- transcript export;
- data backup/restore strategy;
- account deletion workflow.

Exit criteria

The user can leave the product with their own URLs, notes, and generated/extracted text where permitted.

Phase 16 - Performance, Cost and Production Hardening

Build

- queue autoscaling strategy;
- caching;
- adapter rate-limit handling;

- database indexes review;
- storage lifecycle optimization;
- AI/provider cost budgets;
- circuit breakers;
- abuse/rate controls;
- production monitoring;
- disaster recovery drill.

Exit criteria

Defined SLOs are measured and known bottlenecks/costs are understood before scaling.

24. Recommended MVP Cut Line

A realistic first public/internal MVP should stop after roughly **Phase 06**, with selected platform metadata adapters.

It should already deliver:

- one-tap Android capture;
- cross-platform saved links;
- Inbox;
- notes;
- collections;
- tags;
- favorites;
- basic metadata;
- search;
- reliable retry/status handling.

Do **not** delay validation of the capture/library experience until transcription, OCR, embeddings, and chat are complete.

25. Testing Strategy

25.1 Unit tests

- URL parsing/normalization;
- platform identification;
- duplicate detection;
- adapter contracts;
- AI schema validation;
- chunking;
- permissions.

25.2 Integration tests

- capture -> DB -> queue;
- adapter -> normalized metadata;
- worker retries;
- storage lifecycle;
- transcript persistence;
- search indexing;
- RAG evidence linking.

25.3 End-to-end tests

Key scenario:

share URL -> Save -> Inbox -> metadata ready -> organize -> search -> open item

Later:

share media URL -> process -> transcript -> semantic search -> Ask -> cited answer

25.4 Platform contract tests

External platform tests should be isolated so API/network changes do not make the whole core test suite unreliable.

Maintain recorded fixtures where terms permit, but periodically execute a small live smoke suite.

26. Acceptance Criteria Examples

Capture story

Given a valid HTTPS URL shared from another Android app, **when** the user selects this application and taps Save, **then** a Content Item is persisted even if metadata retrieval is unavailable.

Duplicate story

Given a URL already saved by the same user, **when** it is saved again, **then** the system identifies the existing Content Item and does not silently create a duplicate.

Failure story

Given an extractor receives HTTP 429/rate-limit response, **when** processing fails, **then** the original item remains saved, the job becomes retryable with backoff, and the UI does not claim the item is fully processed.

Ask story

Given semantic search returns no sufficiently relevant saved evidence, **when** the user asks a library question, **then** the system states that the library does not contain enough evidence instead of fabricating an answer.

27. Data Retention Strategy

Default recommendation:

- URL/metadata: until user deletes item/account;
- user notes: until user deletes;
- generated text: until user deletes or reprocesses;
- temporary media: delete automatically after downstream processing unless user explicitly enables retention and it is permitted;
- logs: short operational retention with sensitive content minimized;
- raw platform payloads: avoid unless needed; expire aggressively.

28. Cost-Control Design

Cost should be measurable per item and per user.

Track:

- media MB processed;
- audio minutes transcribed;
- OCR frames;
- AI input/output tokens;
- embeddings generated;
- object storage GB-days;
- provider API calls.

Controls:

- deduplicate processing;
- cache metadata;
- never re-transcribe unchanged media without reason;

- prompt/version-aware regeneration;
- sample OCR frames intelligently;
- batch embeddings;
- allow user tiers/quotas later.

29. Accessibility and Usability

- Minimum touch target sizes suitable for mobile.
- Support system font scaling.
- Do not encode processing state with color alone.
- Provide text labels/icons.
- Ensure transcript and summary can be copied/exported where rights allow.
- Support dark/light theme later; do not let theme architecture block it.
- Make capture usable with one hand and few taps.

30. Visual Design Direction

The product should feel like a **calm research notebook**, not another social feed.

Recommended characteristics:

- clean neutral surfaces;
- strong typography;
- thumbnails as supporting context, not dominant entertainment tiles;
- status chips used sparingly;
- search prominent;
- collection organization obvious;
- minimal motion;
- no autoplay in Library;
- no infinite feed mechanics designed to increase consumption.

Primary design goal: **retrieval and comprehension**.

31. Versioning and Migration Rules

- All database changes via migrations.
- Never manually change production schema without migration record.
- Prompt templates have version IDs.
- Extractor adapters have implementation versions.
- Artifacts retain provider/model/version/provenance.
- API breaking changes use versioning or coordinated migration.
- Store `app_version` and `backend_version` in diagnostics.

32. AI Coding Agent Guardrails - Mandatory

Every AI/developer working on this project must follow these rules:

1. Read this master specification before implementing a phase.
2. Do not invent features not listed in the current phase.
3. Do not remove existing features to simplify a new task without explicit authorization.
4. Do not replace the technology stack silently.
5. Do not hard-code a platform scraper into core domain services.
6. Verify current official platform documentation before building an adapter.
7. Never claim an integration works without tests or an explicitly documented untested status.
8. Never fabricate transcripts, timestamps, captions, metadata, API responses, quotas, or platform permissions.
9. Null/unknown is preferable to invented data.
10. Keep user source data separate from AI-generated interpretation.
11. Run migrations, lint, tests, and build checks required by the repository before marking a phase complete.

12. Report exactly what was changed, what was tested, what was not tested, and what remains unresolved.
13. Preserve backward compatibility unless the phase explicitly introduces a migration.
14. No secrets in source control, logs, screenshots, or generated reports.
15. Any uncertainty that affects architecture must be documented in an ADR instead of silently guessed.

33. Per-Phase Agent Execution Template

Every implementation instruction should use this format:

PHASE: <number and name>

SOURCE OF TRUTH: Reels_Links_to_Content_Master_Spec_v1.0.md

OBJECTIVE

<one clear objective>

IN SCOPE

- ...

OUT OF SCOPE

- ...

PRECONDITIONS

- ...

FILES/COMPONENTS EXPECTED TO CHANGE

- ...

IMPLEMENTATION REQUIREMENTS

1. ...

2. ...

DATA/MIGRATION CHANGES

- ...

SECURITY REQUIREMENTS

- ...

TESTS REQUIRED

- ...

ACCEPTANCE CRITERIA

- ...

DO NOT

- invent missing APIs
- silently change scope
- delete working functionality

FINAL REPORT REQUIRED

- changed files
- migrations
- tests run + results
- build/lint result
- unresolved issues
- exact next phase/readiness state

34. Architecture Decision Records Required

Create ADRs for decisions that would otherwise invite later confusion, including:

- ADR-001: repository structure;
- ADR-002: Flutter mobile architecture/state management;
- ADR-003: backend framework;
- ADR-004: Supabase vs self-managed PostgreSQL responsibilities;
- ADR-005: job queue choice;
- ADR-006: object storage provider;
- ADR-007: transcription provider abstraction;
- ADR-008: AI provider abstraction;
- ADR-009: embedding model/provider;
- ADR-010: media retention policy;
- ADR-011: platform adapter compliance strategy;
- ADR-012: search ranking architecture.

Each ADR must include context, decision, alternatives considered, consequences, and date.

35. Suggested Repository Structure

```
reels-links-to-content/
|- apps/
|  |- mobile_flutter/
|  `-- web/
|- services/
|  |- api/
|  `-- worker/
|- packages/
|  |- domain/
|  |- platform_adapters/
|  `-- shared_contracts/
|- database/
|  |- migrations/
|  `-- seed/
|- docs/
|  |- Reels_Links_to_Content_Master_Spec_v1.0.md
|  |- adr/
|  `-- adapters/
|- tests/
|  |- integration/
|  `-- fixtures/
|- scripts/
|- .env.example
`-- README.md
```

Actual structure may differ after ADR-001, but separation of domain, adapters, and infrastructure must be preserved.

36. Example Content Item JSON

```
{
  "id": "uuid",
  "original_url": "https://...",
  "canonical_url": "https://...",
  "platform": "tiktok",
  "content_type": "short_video",
  "title": "Example title",
  "creator": {
    "name": "Creator Name",
    "handle": "creator"
  },
  "status": "ready",
  "saved_reason": "learn",
}
```

```

"user_note": "Useful architecture idea for my project",
"favorite": false,
"collection": "Programming",
"source": {
  "availability": "available",
  "metadata_method": "official_oembed"
},
"artifacts": {
  "transcript": {"status": "ready", "artifact_id": "uuid"},
  "ocr": {"status": "not_run"},
  "summary": {"status": "ready", "artifact_id": "uuid"}
},
"captured_at": "2026-09-27T15:00:00+03:00"
}

```

37. Future Expansion Without Architectural Rewrite

The `ContentItem` abstraction should eventually accept:

- short-form social videos;
- standard YouTube videos;
- web articles;
- X posts/threads;
- Reddit posts;
- PDFs;
- podcast episodes;
- images/screenshots;
- manually entered notes.

Do not add every type in MVP UI. The architecture should simply avoid assuming that all content is a reel or all extracted knowledge comes from audio.

38. Risks and Mitigations

Risk: platform scraping breaks frequently

Mitigation: official APIs first, adapter isolation, link-only fallback, observability.

Risk: product becomes an expensive AI pipeline before user value is proven

Mitigation: launch capture/library MVP before transcription and RAG.

Risk: media storage cost explodes

Mitigation: temporary processing storage by default, lifecycle deletion, object storage metrics.

Risk: AI summaries make unsupported claims

Mitigation: preserve source artifacts, evidence-linked outputs, schema validation, user-visible generated labels.

Risk: semantic search feels inaccurate

Mitigation: hybrid search, source-aware weighting, evaluation set, visible match context.

Risk: users cannot trust processing state

Mitigation: explicit stage status, partial-success model, retryable errors.

Risk: unofficial extraction violates terms or becomes operationally fragile

Mitigation: adapter documentation, compliance review, do not treat unofficial scraping as a foundational dependency.

39. Product Success Metrics

Early product metrics should focus on utility, not attention:

- captures per active user;
- percentage of saves successfully retrievable later;
- search success rate;
- percentage of items organized automatically/manually;
- time from share tap to confirmed save;
- processing success by platform;
- search-to-open conversion;
- repeat retrieval of older items;
- user corrections to AI tags/collections;
- cost per processed item/minute.

Avoid optimizing for endless time-in-app.

40. Launch Checklist

Before MVP release:

- ☐ Auth security tests pass.
- ☐ Android share target works from at least browser + two major social apps.
- ☐ Link capture works when offline/poor network if local queue is included, or failure behavior is clear.
- ☐ Duplicate handling tested.
- ☐ Delete item works completely.
- ☐ Collection/note/favorite flow works.
- ☐ Processing failures do not lose URLs.
- ☐ Logs contain no secrets.
- ☐ Platform adapter docs include verification dates.
- ☐ Database backups configured.
- ☐ Privacy policy reflects real data behavior.
- ☐ No unsupported claim that every reel can be downloaded/transcribed.
- ☐ Search returns predictable results.
- ☐ Build/version shown in diagnostics.

41. Immediate Next Build Sequence

The implementation should begin in this order:

1. Commit this document as `/docs/Reels_Links_to_Content_Master_Spec_v1.0.md`.
2. Create ADR-001 through ADR-005 before writing platform extractors.
3. Implement Phase 01 application/auth shell.
4. Implement Phase 02 share capture and durable Content Item storage.
5. Validate the capture UX with real daily use.
6. Implement Phase 03 organization and Library.
7. Build the job framework.
8. Only then invest in external platform adapters.

The strongest validation milestone is not “AI can summarize a reel.” It is:

The user can save something instantly, forget about it, and reliably find it later.

42. Current External-Integration Verification Notes

This section records only the high-level facts verified while creating version 1.0. It is not a substitute for adapter-specific implementation research.

- **Android:** official Android documentation describes receiving shared text/URLs via `ACTION_SEND` and manifest intent filters; Compose Navigation can route shared content into a target screen.
- **TikTok:** official TikTok developer documentation describes `oEmbed` for video URLs and a Display API for authorized user/profile/video metadata. TikTok indicates app setup/permissions may be required for Display API use.
- **YouTube:** the official YouTube Data API provides video and channel resources and requires an API key or OAuth token depending on the request/authorization needs.

Before implementation, record the official URLs and verification date in the relevant adapter document. If documentation or terms conflict with this specification, **do not guess**: stop the adapter implementation, document the discrepancy, and update the relevant ADR/spec with an explicit decision.

43. Change Control

This file is versioned. Future edits must update:

- version number;
- date;
- change summary;
- affected phases/data models/API contracts.

Do not silently rewrite the master specification to match accidental implementation. Implementation should follow the specification unless an intentional product/architecture decision changes the specification first.

Version History

Version	Date	Status	Summary
1.0	2026-09-27	Implementation Baseline	Initial full product, UX, architecture, data, security, testing and phased implementation specification.

Appendix A - Definition of Done for Any Phase

A phase is complete only when all applicable conditions are satisfied:

- scope implemented;
- migrations included and reversible/forward-safe as designed;
- unit/integration tests pass;
- lint/build/type checks pass;
- security implications reviewed;
- logs/metrics added for new critical processing;
- UI includes loading, empty, success, partial, and failure states;
- no secrets committed;
- documentation updated;
- unresolved issues explicitly listed;
- next phase entry criteria known.

Appendix B - Agent Handoff Summary

When handing this project to a new AI agent or developer, provide:

1. this master specification;
2. latest repository commit/branch;

3. environment/setup README;
4. current database migration number;
5. completed phase list;
6. current unresolved issues;
7. adapter verification documents;
8. test/build status;
9. exact next phase instruction.

The handoff should never rely on chat history alone.

Appendix C - One-Sentence Guardrail

Save the user's link first; enrich it honestly; keep platform integrations replaceable; preserve provenance; never invent missing source data.

Appendix D - Verified Official Reference Links (27 September 2026)

These references were checked while preparing version 1.0. Re-verify them before implementing a platform adapter because permissions, endpoints, quotas, review requirements, and terms can change.

- Android - Receiving simple data from other apps: <https://developer.android.com/develop/ui/compose/sharing/receive>
- Android - Sharing data between apps overview: <https://developer.android.com/develop/ui/compose/sharing>
- TikTok - Display API overview: <https://developers.tiktok.com/doc/display-api-overview/>
- TikTok - Embed videos / oEmbed: <https://developers.tiktok.com/doc/embed-videos/>
- TikTok - Display API get started: <https://developers.tiktok.com/doc/display-api-get-started/>
- YouTube - Data API reference: <https://developers.google.com/youtube/v3/docs>
- YouTube - Data API getting started: <https://developers.google.com/youtube/v3/getting-started>

Implementation rule: if a reference has moved, is unavailable, or no longer authorizes the intended behavior, do not substitute an unofficial method silently. Document the change and make an explicit architecture/product decision.